Does Deterministic Generate–Validate–Repair Reduce Unsafe LLM-Generated Network Changes? A Confirmatory Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a sealed paired confirmatory study (n = 200 intent→configuration tasks) to test whether a deterministic generate–validate–repair (GVR) pipeline that consults a deterministic NetOps validator reduces validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline. The run used a single hosted model (gpt-5-mini) with adapter-enforced shared-initial generation semantics and a primary deterministic validator (deterministic_netops_validator_v5). Execution completed all planned pairs (completed_pairs = 200) consuming 200 model calls (one shared initial generation per task); no repair calls were issued because every initial candidate passed the validator. Paired contingency: n11 = 200, n10 = 0, n01 = 0, n00 = 0; estimate = 0.0; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.0, 0.0] (10,000 resamples, seed = 1729). Because the baseline validator-detected unsafe incidence was 0% on this task bank, the preregistered >=50% relative-reduction hypothesis could not be meaningfully evaluated. We report the sealed design, execution accounting (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z), the Mininet 10% holdout (20 tasks) that produced zero validator–emulator disagreements, and an artifact-grounded discussion of operational implications and threats to validity (preregistration id: cnsms2026-gvr-effectiveness-02).**

I. Introduction

Automating intent→configuration translation with generative language models can increase NetOps productivity but risks producing incorrect or unsafe changes that cause outages or policy violations. Prior work therefore proposes grounding, deterministic validation, and repair loops (generate–validate–repair, GVR) to detect and correct unsafe candidates before application. The operational benefit of automated repair is conditional on three factors: (i) baseline prevalence of validator-detectable failures from the chosen generator/prompting policy; (ii) validator coverage and fidelity; and (iii) the available repair budget and repair-prompt effectiveness. We preregistered a narrowly scoped confirmatory question: Does an automated, deterministic GVR pipeline that consults a deterministic NetOps validator materially reduce validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline on a curated 200-task intent→configuration bank? The study was designed as a paired experiment that fixes the generator output per task and isolates the marginal effect of invoking a validator+repair on the same candidate.

II. Related Work

Surveys and empirical studies across LLM applications document factual errors, hallucinations, prompt sensitivity, and non-determinism; mitigation approaches include retrieval grounding, verifier-assisted repair, and constrained agent architectures. NetOps prototypes show feasibility of intent→configuration generation but typically lack instrumented, production-like failure-rate evaluations. Architectural proposals recommend combining LLM generation with deterministic validators (e.g., Batfish-style checkers) and with guarded agent patterns; red-team audits demonstrate configuration-level brittleness and tool-description injection vectors that can alter safety outcomes. Our study positions itself as an externally auditable, preregistered measurement that isolates the marginal effect of deterministic repair for a fixed model and sampling policy, addressing gaps about when GVR actually reduces validator-detected failures in practice.

III. Methodology

Preregistration and inferential plan. We froze the experimental plan under preregistration id cnsms2026-gvr-effectiveness-02 before any model calls. The confirmatory design was paired and deterministic: n = 200 tasks (four topology clusters × 50 tasks), one shared deterministic initial LLM generation per task, and a repair budget of at most one automated repair call per task. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline and the prespecified test was an exact two-sided McNemar test (α = 0.05). Confidence intervals for the paired difference were computed by the bootstrap percentile method with 10,000 resamples and bootstrap_seed = 1729. The analysis executor was paired_binary_analysis_v1 as recorded in the analysis artifacts.

Adapter, model, and sampling semantics. The run used a single hosted model declared as gpt-5-mini (provider label: openai_responses). Model configuration metadata (execution/model_configuration.json) records requested_model = "gpt-5-mini", max_output_tokens = 2000, maximum_attempts_per_call = 1, and temperature = null (deterministic sampling enforced by the adapter). Crucially, the hosted_netops_gvr_v1 adapter enforced shared_initial generation semantics: for each task the system generated exactly one initial candidate which was deterministically reused for both

the baseline (LLM-only) and guarded (GVR) conditions. This shared-initial design eliminates per-condition sampling variance and isolates the incremental effect of invoking validation and an allowed repair step on the identical generator output. Adapter accounting in deterministic_reconciliation.json documents hosted_model_call_event_count = 200, stage_counts = {shared_initial_generation: 200}, and model_calls_used = 200, confirming the enforced single shared invocation per task and that no separate condition-specific sampling occurred.

Task bank, validators, and holdout emulator. The task bank is a curated, machine-checkable intent→configuration set (200 tasks) in which each task encodes initial and expected final states plus workflow policies (e.g., make_before_break_uplink_switchover, must_be_down_for_fields). The primary deterministic checker (deterministic_netops_validator_v5, validator_version = "5.0") runs syntactic parsing, intent/constraint enforcement, and structured diagnostics (fields available in validator traces include valid flag, violation_count, violation_codes, parsed_command_count, required_command_count, and difficulty annotations). A Mininet-based data-plane emulator was preregistered as a secondary disagreement detector applied to a randomized 10% holdout (20 tasks) to detect validator–emulator mismatches; the executed contamination_summary artifact reports zero disagreement flags for that holdout. The preregistration specified deterministic treatment rules: pairs flagged for validator–emulator disagreement would be excluded from the primary analysis per the contamination_plan; sensitivity analyses for alternative treatments were preregistered but not invoked because no flags occurred.

Repair policy, budgeting, and missingness handling. Per the preregistration, when the primary validator reported violations the pipeline was permitted to issue up to one automated repair prompt to the same LLM for that task (maximum repair calls across the sample = 200). Repair prompts, if used, would be captured in repair_provider_trace fields in the episode artifacts; none appear in the archived episodes because no repairs were applied. Missingness and exclusion rules were deterministic: pairs where neither condition completed because of infrastructure failures would be excluded and logged; pairs with contamination flags (validator–emulator disagreement) would be excluded from the primary confirmatory test. The execution manifest and missingness_summary show complete_pair_count = 200 and zero failed episodes, so the planned missingness handling was not invoked in practice.

Execution accounting and artifact capture. The autonomous run executed between 2026-08-30T11:50:34.530059Z and 2026-08-30T12:12:07.541650Z. The adapter enforced the preregistered maximum model calls (maximum_model_calls = 400) and per-task caps (initial generation = 1 call; repair calls <= 1). The run consumed 200 model calls (one shared initial generation per task), with cache_miss_count = 200 and cache_hit_count = 0 as recorded in the deterministic reconciliation artifacts. All model calls, prompts, raw responses, validator traces, provider traces, and analysis artifacts (execution/raw_results.jsonl, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/contamination_summary.csv, analysis/analysis_log.jsonl, execution/model_configuration.json) were archived; representative per-task artifacts (e.g., execution/scoring/task-000001-baseline.json) show validator_trace fields and scoring outcomes. The analysis pipeline executed the exact McNemar test and bootstrap CI generation exactly as preregistered; analysis/analysis_log.jsonl records bootstrap_resamples = 10000 and bootstrap_seed = 1729.

Design rationale and implications for internal validity. The shared_initial paired design was chosen to isolate the pure marginal effect of validation+repair on a fixed generator output; this reduces variance attributable to stochastic sampling and makes the confirmatory test focused on whether repair can convert a validator-failing candidate into a validator-passing one for the same initial output. The trade-off is explicit: the design sacrifices generality across multiple independent generator draws for stronger internal control. Preregistration, deterministic adapter semantics, and frozen stopping rules were used to avoid post-hoc analytic choices. Deterministic reconciliation artifacts confirm pair_accounting_consistent = true and all_deterministic_consistency_checks_passed = true.

Limitations baked into the methodology. The methodology intentionally restricts scope: a single model, a deterministic prompt/sampling policy, a one-repair budget, and a curated task bank. These constraints reflect an explicit confirmatory question about marginal GVR effect under a tightly controlled generator policy rather than an evaluation of alternative sampling, multi-step repair strategies, or cross-model generality. When interpreting outcomes, readers should account for these preregistered methodological choices documented in the archived plan.

IV. Results

Execution completed all planned episodes and pairs. Accounting: completed_episode_count = 400 (shared initial generation reused across conditions), complete_pair_count = 200, model_calls_used = 200 (one shared initial generation per task), maximum_model_calls = 400, repair_calls_used = 0. Adapter/provider audit artifacts report hosted_model_call_event_count = 200, completed_hosted_model_call_event_count = 200, failed_hosted_model_call_event_count = 0, cache_miss_count = 200, and stage_counts = {shared_initial_generation: 200}, confirming the preregistered single shared initial invocation semantics.

Validator outcomes and paired contingency. Every initial candidate passed the primary deterministic validator: baseline_success_count = 200/200 and guarded_success_count = 200/200 (analysis/condition_summary.csv). The paired contingency table is n11 = 200, n10 = 0, n01 = 0, n00 = 0 (analysis/paired_contingency_table.csv). Because there are no discordant pairs, the observed paired difference estimate = 0.0. The prespecified exact McNemar two-sided test yields

p = 1.0. Bootstrap percentile confidence intervals (10,000 resamples, bootstrap_seed = 1729 as recorded in analysis/analysis_log.jsonl) produce a 95% CI = [0.0, 0.0]. Under the preregistered estimand and test the preregistered >=50% relative-reduction hypothesis is not evaluable on this sample because baseline validator failures are absent.

Repair and secondary outcomes. No validator failures were observed; consequently, the preregistered repair branch was never invoked and secondary_results are empty in the analysis bundle. The planned secondary estimands (average repair iterations per repaired task, GVR-introduced failure rate, model-call cost per successful repair) therefore have no empirical data in this run and are recorded as empty artifacts. The analysis/contamination_summary.csv (the preregistered Mininet holdout check) contains no flagged disagreements: the randomized 10% Mininet holdout (20 tasks) produced zero validator–emulator mismatches per that artifact.

Representative diagnostics and task difficulty. Representative per-task validator traces (e.g., execution/scoring artifacts for pair-000001...pair-000006) confirm that the shared initial generator outputs matched the task required_sequence and that validator_trace.validation_after.valid = true with violation_count = 0. These representative artifacts also show that the task bank exercised nontrivial workflow constraints: difficulty labels in the archived validator traces include level values documented as "medium", "hard", and "very_hard" and patterns such as make_before_break_uplink_switchover, paired_link_atomic_mtu_change, and rolling_access_vlan_migration. These labels demonstrate the benchmark included tasks intended to exercise complicated dependency constraints despite producing validator-satisfying outputs under the chosen generation policy.

Missingness, exclusions, and contamination. There were no technical failures affecting the primary analysis: missingness_summary records complete_pairs = 200 and zero failed or unscorable episodes. The contamination detector executed on the preregistered 10% randomized holdout (20 tasks) and found zero disagreements (analysis/contamination_summary.csv). We note a documented reproducibility gap: the explicit randomized selection seed for the Mininet holdout is not labeled in the archived artifacts, which prevents exact deterministic reconstruction of which tasks comprised that holdout despite the contamination summary reporting zero disagreements.

Unexecuted sensitivity branches. Planned sensitivity analyses for alternative contamination treatments and missingness handling were not invoked because no contamination flags or execution failures occurred. Because no repair attempts occurred, we could not empirically evaluate GVR-introduced failure rates or repair-iteration distributions; correspondingly, the preregistered exploratory analyses returned empty results and are recorded as such in the analysis artifacts.

V. Discussion

Conditional interpretation. The degenerate null outcome demonstrates a logical constraint: when the chosen generator, prompt template, and sampling policy produce validator-passing candidates for every item in a workload, an added validator-coupled repair step cannot reduce validator-detected failures. This artifact-grounded observation clarifies that GVR's marginal utility is a function of baseline validator failure prevalence and validator coverage rather than a universally guaranteed safety gain.

Task and prompt alignment. Archived validator traces and representative task artifacts show that tasks encoded explicit required_sequences and workflow constraints, and that the deterministic prompts and adapter-enforced shared-initial semantics produced outputs conforming to those constraints. Because the task bank was curated with machine-checkable intent representations, the generator had a narrow, high-signal mapping from intent to commanded sequence—reducing semantic ambiguity and making validator-passing outputs more likely. Difficulty annotations (e.g., patterns labeled make_before_break_uplink_switchover or paired_link_atomic_mtu_change) indicate that the workload exercised nontrivial coordination constraints; the absence of validator failures therefore reflects alignment between task formalization, prompt semantics, and validator checks rather than trivial task simplicity.

Validator fidelity and emulator coverage. Deterministic_netops_validator_v5 enforces a defined set of syntactic and intent constraints; the preregistered Mininet secondary check (10% holdout = 20 tasks; see analysis/contamination_summary.csv) executed and reported zero disagreements, supporting the internal consistency of validator verdicts on the sampled holdout. Nevertheless, the limited scope of the holdout and the disclosed absence of an explicit labeled seed for holdout selection constrain claims about broader emulator coverage: the artifact bundle shows no validator–emulator mismatches for the tested subset, but this does not prove absence of validator blind spots across all production conditions. Practitioners should therefore treat validator pass rates as conditional on validator completeness and the representativeness of any emulation checks used to validate that completeness.

Design choices that shaped outcomes. The preregistered shared_initial paired design intentionally removes per-condition resampling as a lever to create discordant cases; it isolates the incremental effect of invoking a repair step on a fixed generator output. That controlled isolation is scientifically valuable for measuring the marginal effect of repair on the exact same candidate, but it also reduces the experimental pathways by which GVR could yield observed improvements in a different design (for example, multi-sample generation where a repair step might solicit alternative formulations). Similarly, the single-model, deterministic sampling policy reduces stochastic variance but limits generality: different sampling temperatures, multi-sample strategies, or model ensembles may produce nonzero baseline failure rates and thus different measured GVR benefits.

Implications for operational design and future studies. The empirical datum—zero baseline validator failures under these

controlled, curated conditions—suggests concrete operational guidance: teams considering automated repair should first instrument representative workloads to measure baseline validator failure prevalence and then evaluate validator completeness via targeted emulation or real-device checks. If baseline validator failures are rare under the planned generator/prompt policy, investments that raise validator fidelity (vendor-specific behavior models, richer device semantics) or that increase input diversity (adversarial testing, multi-sample generation, multi-model ensembles) may provide greater marginal safety returns than adding an automated repair stage with a small repair budget. Conversely, when baseline failures are nontrivial, a well-engineered GVR pipeline with adequate repair budget and high-quality repair prompts can reduce validator-detected issues; future experiments should measure repair iterations and model-call cost per successful repair when such failures are present.

Recommended experimental refinements. To quantify GVR effectiveness in settings where it can act, future confirmatory runs should (a) target task banks or prompt variants with nonzero baseline validator failures (for example, by introducing realistic ambiguity or controlled perturbations), (b) evaluate multi-sample generation and multi-model ensembles as baselines, (c) increase repair budgets and test multi-step repair strategies including human-in-the-loop handoff, and (d) expand emulator or real-device coverage and record explicit, reproducible holdout selection seeds so that contamination checks are fully reconstructable. The archived artifacts show that secondary_results were empty because no repairs occurred; capturing repair-cost and iteration metrics requires a design where repairs are invoked in practice.

Threats to validity revisited. Construct validity: validator pass/fail outcomes reflect the formalized constraints encoded in deterministic_netops_validator_v5; absent richer device modeling, some operationally relevant failures may not be detected. External validity: a single hosted model (gpt-5-mini), deterministic sampling, shared_initial semantics, and a curated task bank constrain generalizability. Internal validity: preregistration and deterministic adapter enforcement strengthen causal interpretation in the paired comparison, but the controlled choices also narrow the space of behaviors observed. Reproducibility: the execution bundle archives model calls, validator traces, and the contamination summary showing zero Mininet disagreements, but the missing labeled Mininet holdout seed prevents deterministic reconstruction of the exact holdout sample—this gap is disclosed in the Disclosure Statement.

Concluding synthesis. The completed artifacts and diagnostic traces make clear that the preregistered GVR pipeline, as instantiated here, produced no measurable reduction in validator-detected failures because there were no such failures to reduce. That operationally informative null result highlights that the safety value of automated repair is not universal but workload- and validator-dependent. Future work should deliberately select or construct workloads with nontrivial baseline failure rates and expand validator fidelity and sampling strategies to empirically quantify when GVR provides practical safety gains.

VI. Limitations

- Task-bank scope: the confirmatory sample used a curated synthetic/public intent→configuration bank (n = 200); results therefore reflect this bank and may not generalize to broader production workloads or adversarial distributions.
- Single-model and shared-initial setting: only gpt-5-mini (hosted) with adapter-enforced shared-initial generation was tested (execution artifacts record hosted_model_call_event_count = 200 and stage_counts shared_initial_generation = 200). Outcomes may differ across model families, independent per-condition sampling, nonzero temperature sampling, or larger repair budgets.
- Validator coverage and oracle limitations: the primary deterministic validator enforces a defined set of syntax/intent constraints; validators can fail to capture real device idiosyncrasies, vendor-specific behaviors, or emergent failure modes detectable only by end-to-end deployment.
- Adversarial robustness untested: this confirmatory run did not include active adversarial injection or red-team tool-description attacks; prior audits indicate such vectors can bypass naive defenses and remain a separate concern.
- Reproducibility gap for contamination check: the randomized holdout selection seed for the Mininet contamination check is absent from the labeled artifacts, preventing exact reconstruction of the holdout selection.
- Single repair budget: the GVR pipeline allowed at most one automated repair prompt per task; multi-step repair strategies, different repair budgets, or human-in-the-loop approvals were not exercised here.

VII. Conclusion

We preregistered and executed a sealed paired study comparing LLM-only generation to a deterministic GVR pipeline on 200 NetOps intent→configuration tasks using gpt-5-mini and deterministic_netops_validator_v5. Both pipelines produced validator-passing configurations for every task (n11 = 200; no discordant pairs), resulting in an observed paired difference of 0.0, exact McNemar p = 1.0, and bootstrap 95% CI = [0.0, 0.0]. The preregistered >=50% relative-reduction hypothesis could not be evaluated because baseline validator failures were absent. The artifact-grounded outcome clarifies that GVR's marginal value is workload dependent: when baseline validator-detectable failures are rare, automated repair yields no measured benefit for those validator-detected errors. Future studies should target workloads with nontrivial baseline failure rates, expand validator fidelity with richer emulation or real-device checks, evaluate multi-sample and multi-model policies, and explore multi-step or human-in-the-loop repair budgets to characterize practical operational gains. All execution artifacts and analysis outputs are archived for independent inspection under the preregistration id cited above.

Disclosure Statement

Disclosure (mandatory). This study executed under the autonomous post-lock preregistration cnsms2026-gvr-effectiveness-02. Declared model: gpt-5-mini (provider label: openai_responses) invoked via the hosted_netops_gvr_v1 adapter. The exact initial master prompt is archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Topic constraints: the human Research Director limited the study to Generative AI and LLMs for NetOps focusing on safety/reliability of generated network changes. Frameworks and tools: orchestration used hosted_netops_gvr_v1 and the deterministic task generator deterministic_netops_task_generator_v5. Primary deterministic validator: deterministic_netops_validator_v5 (intent/constraint checks). A Mininet-based emulator was preregistered and executed as a secondary disagreement detector on a randomized 10% holdout (20 tasks); the artifact analysis/contamination_summary.csv shows zero disagreements for that holdout. Preregistration and freeze: the experimental plan (estimand, sample size n = 200, paired design, stopping rules, max model calls = 400, repair budget = 1 per task, shared-initial generation semantics) was frozen before any model calls. Autonomous execution and artifacts: the run executed autonomously under the frozen plan (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z); model calls, prompts, seeds, validator traces, emulator traces, logs, and the artifact manifest are archived in the execution bundle. Model-call budget and usage: 200 model calls were used (one shared initial generation per task); up to 200 repair calls were allowed but none were applied because initial candidates passed the validator. Human involvement: the human Research Director selected the topic family and pre-lock constraints; no human manual editing or selective rewriting of technical manuscript content occurred after the autonomy lock. Deviations and restarts: no post-preregistration design changes or technical restarts occurred. Known reproducibility gap: the explicit randomized selection seed used to draw the Mininet 10% holdout is not present as a labeled artifact in the archive; this absence prevents exact deterministic reconstruction of the drawn holdout despite the contamination summary reporting no disagreements. The master prompt archive reference above is the immutable prompt required by the track.

References

[1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

[2] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[3] Oscar G. Lira, Oscar Maurício Caicedo Rendón, Nelson L. S. da Fonseca, "Large Language Models for Zero Touch Network Configuration Management", 2024, 10.1109/mcom.001.2400368

[4] Chirag Shah, Ryen W. White, Reid Andersen, Georg Buscher, Scott Counts, Sarkar Snigdha Sarathi Das, Ali Montazeralghaem, Sathish Manivannan, J. Neville, Nagu Rangan, Tara Safavi, Siddharth Suri, Mengting Wan, Leijie Wang, Longqi Yang, "Using Large Language Models to Generate, Validate, and Apply User Intent Taxonomies", 2025, 10.1145/3732294

[5] omar altawil, Dr. Mustafa Al-Fayoumi, "Configuration-Level Semantic Brittleness in Tool-Using LLM Agents: A Factor-Ranking Study of Persona, History, and Tool Definition", 2026, 10.2139/ssrn.7203631

[6] Mohammadreza Rashidi, "Measuring How LLM Tool Descriptions, Cross-Tool Ambiguity, and Action Chains Compose into Excessive Agency Exploits", 2026, 10.21203/rs.3.rs-10646209/v1

[7] Yoshihiro Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents", 2026, 10.36227/techrxiv.177006109.97957916/v1

[8] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[9] Shuyin Ouyang, Jie M. Zhang, Mark Harman, Meng Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation", 2024, 10.1145/3697010

[10] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1